Лабораторна робота №5

Тема: Асинхронна обробка

Як розробник, що підтримує цей код, я хочу, щоб ці операції виконувалися в окремо налаштованому пулі потоків із обмеженою кількістю ресурсів, а збої та перевантаження оброблялися передбачувано, щоб «асинхронність» не означала «непомітні збої» чи «вичерпання ресурсів сервера».

**Опис**:

У власному проєкті (в межах доменної логіки) знайдіть 3 приклади повільних побічних ефектів, які наразі блокують викликаючий код: надсилання електронного листа, генерування PDF-файлу, виклик зовнішнього сервісу чи будь-яку іншу дію, що триває близько секунди.

**Реалізація**:

- «Запустив і забув» (Fire-and-forget).
- Результат, необхідний викликаючому коду: пара незалежних повільних викликів (кожен — це метод із @Async, що повертає CompletableFuture<T>), результати яких викликаючий код об’єднує (за допомогою `thenCombine` або `allOf`, якщо викликів більше двох) в один підсумковий результат.
- Обробка виняткових ситуацій: збій асинхронного виклику не повинен призводити до “падіння” програми.

**Сценарій**: *Повільний метод більше не блокує викликаючий код*

**GIVEN** доменна операція (викликаючий код), що викликає повільний метод

**WHEN** операцію викликано

**THEN** вона повертає керування до завершення виконання повільного методу

**AND** повільний метод завершується пізніше, у потоці вашого власного виконавця (executor)

**Сценарій**: *Виконавець є явно визначеним і має обмежені ресурси*

**GIVEN** бін `ThreadPoolTaskExecutor` із явно заданими розмірами пулу (базовим і максимальним),

місткістю черги та префіксом назви потоку

**WHEN** виконується метод із анотацією @Async

**THEN** він працює в потоці, назва якого починається з цього префікса

**AND** не в пулі за замовчуванням Spring Boot чи в SimpleAsyncTaskExecutor

**Сценарій**: *Два незалежні виклики справді виконуються одночасно*

**GIVEN** два методи з @Async, що повертають CompletableFuture<T>

**WHEN** викликаючий код запускає обидва методи, не чекаючи на завершення жодного з них, а потім об'єднує їхні результати

**THEN** викликаючий код отримує об'єднаний результат

**AND** тест підтверджує, що обидва виклики виконувалися одночасно

**Сценарій**: *Невдалий асинхронний виклик не призводить до збою викликаючого коду*

**GIVEN** асинхронний метод, який кидає exception

**WHEN** викликаючий код обробляє отриманий Future

**THEN** викликаючий код продовжує роботу, використовуючи механізми exceptionally або handle і запит не завершується помилкою

**AND** для методу @Async із типом повернення void обробник AsyncUncaughtExceptionHandler фіксує помилку, а не приховує її.

**Сценарій**: *Очікування завжди обмежене в часі*

**GIVEN** місце, де викликаючий код очікує на завершення Future

**WHEN** виконання Future триває занадто довго

**THEN** викликаючий код припиняє очікування через тайм-аут.

**Сценарій**: *Переповнення виконавця обробляється цілеспрямовано*

**GIVEN** виконавець, спеціально налаштований із дуже малим пулом і чергою

**WHEN** надсилається більше завдань, ніж він може вмістити

**THEN** виняток TaskRejectedException, що виникає внаслідок цього, обробляється згідно з явним рішенням (повторна спроба пізніше, виконання в потоці викликаючого коду або відповідь із чітким повідомленням «спробуйте пізніше»)

**AND** він НЕ призводить до випадкової помилки 500

**AND** тест демонструє сценарій відхилення завдання